问题一模型建立与求解

问题一分析

问题一要求由化学组成预测多维感官强度。本题可用样本为 250 款啤酒，输入包含 231 项化学与理化特征，监督信号为 50 个标准化连续感官属性。其困难在于输入维数接近样本量、化合物之间相关性较强，各感官家族的可预测性又不相同；附件中的标准化值还不能唯一还原为评委原始 0—6 分。风格分布不均，单一平均误差也可能掩盖缺陷味及小样本风格的失败。

本文按 beer 字段对齐附件，在训练折内完成填补与特征变换；以 ExtraTrees 为非线性基线，将 HGB、秩高斯 Ridge 和 PLS2 的互补预测纳入非负最小二乘堆叠，并通过感官组向共享权重收缩控制方差。采用 10 次重复、每次 3 折的外层验证，基学习器选择与堆叠权重估计均限制在外层训练数据内。评价同时覆盖解释度、绝对误差、排序一致性及弱目标、风格误差，0—6 仅按训练折经验分布给出相对强度。

问题一模型

数据口径与缺失值插补

主键使用 beer，beer_id 存在精度问题，不参与连接或建模；风格字段仅用于分层验证及分组误差报告，不作为最终模型输入。表 1 给出预处理步骤与维度变化。所有需估计的填补、变换及映射参数只由相应训练折计算。附件 1 的给定尺度保持不变，已确认的 log10 字段不重复取对数；其中负值表示原浓度小于 1。

表 1 数据预处理步骤与维度变化

步骤	处理与范围	维度变化	理由
读取核对	附件 1 与附件 4	250×234；250×53	核对主键、目标与字段类型
样本对齐	按 beer 一一连接	250+250→250 行	避免标识精度导致错配
字段拆分	剔除三列元数据	X：250×231；Y：250×50	排除身份与风格标签入模
尺度保留	不重复取 log10、不全局截尾	250 行、231 输入不变	保留实际差异与极端观测
折内插补	训练中位数与缺失指示	231→231+k，0≤k≤9	73 单元、63 行、9 列缺失
分支变换	线性分支折内秩高斯变换	样本及目标数不变	减轻偏态、共线与量纲影响

步骤	处理与范围	维度变化	理由
相对映射	仅用训练折经验分布	50 维连续值＋50 维相对等级	不声称恢复原始评委均分

全部 250 行均保留，不按全数据分位数删除异常样本。37 个 log10 字段及 33 个原尺度字段可由附件 8 识别，但这不意味着两附件所有对应单元格一致；发现冲突时保留输入原值并单列核验，不静默替换。缺失机制及跨批次可比性属于模型假设，需由后续外部样本检验。

连续监督信号与相对标签

以标准化感官向量作为监督信号，多输出关系定义为式(1)。每款啤酒的化学特征向量记为 x，感官向量记为 y；误差项包含未测化合物、评委差异及随机扰动。

$$y_i = F(x_i) + \varepsilon_i, \qquad F: \mathbb{R}^{231} \to \mathbb{R}^{50}, \qquad i = 1,\ldots,250 \tag{1}$$

附件 4 包含标准化后汇总值，缺少完整的评委中心与尺度参数，因此从它到原始 0—6 均分并非唯一。连续尺度承担模型训练与误差评价，相对等级仅承担展示；两者分开报告。

基学习器与特征构造

基学习器由 ExtraTrees、直方图梯度提升 HGB、秩高斯 Ridge 和秩高斯 PLS2 组成。ExtraTrees 刻画随机化非线性与交互[2]，HGB 补充局部阈值响应，Ridge 缓和共线，PLS2 提供多输出共享的低秩表示。树分支保留附件尺度；线性分支在各训练折中填补后进行秩高斯变换，并将缺失指示单独并入。表 2 列出完整候选范围。

表 2 基学习器与折内候选参数

学习器	候选范围	数据处理
ExtraTrees	300 棵；特征比例 0.33/0.60/1.00；叶节点 1/2/4/8；不限深度	中位数填补＋缺失指示
HGB	叶节点上限 7/15；最小叶样本 20；学习率 0.05；100 轮；L2=1	中位数填补＋缺失指示
Ridge	正则强度 0.1/1/10/100/1000	折内秩高斯变换与缩放
PLS2	潜变量数 2/5/10/15/20	折内秩高斯变换与多输出拟合

内层以宏平均 MAE 最小选择每个学习器的参数；相同时依次比较 R^2、Spearman 和两项七档代理指标。候选范围固定，外层验证数据不参与参数、权重或截距估计。HGB 候选也可能得到相同预测，本文保留固定候选范围，不据此扩大搜索。

感官组收缩非负堆叠模型

堆叠利用训练数据内部的交叉验证预测估计组合系数，其非负约束可减少相关预测器之间的相互抵消[1]。设 T 为当前外层训练样本，z 为各基学习器在 T 上经内层 3 折得到的折外预测，m 为四个基学习器的索引。对目标集合 G，分别求解式(2)；G 可取全部目标或某一感官家族。

$$w_G^{*}=arg\ \ min_{w\ge 0}\sum_{j\in G}\sum_{i\in T}\left(\frac{y_{ij}-\bar{y}_j-\sum_{m=1}^{4}w_m\left(z_{ijm}-\bar{z}_{jm}\right)}{s_j}\right)^2 \tag{2}$$

式中，y 的横线为训练目标均值，z 的横线为对应内层折外预测均值，s 为训练目标的标准差；零方差时取 1。标准差归一化使高波动目标不独占权重拟合。按香气 15 维、风味 15 维、其他味觉 15 维、口感 4 维、总体喜好 1 维分组；组别由字段定义预先确定，与啤酒风格不同。

为降低小感官组的权重方差，将组内解向全目标共享解收缩，定义式(3)。固定 λ=0.5，不使用外层指标选择收缩强度。

$$\widetilde{w}_g=\left(1-\lambda\right)w_g^{*}+\ \lambda w_{全部}^{*},\qquad \lambda\ =\ 0.5 \tag{3}$$

非负约束只要求各权重不小于 0，不要求权重之和为 1。每个目标保留自由截距，按训练均值恢复为式(4)，因此截距可为负值，不能与权重一同约束为非负。

$$b_j=\bar{y}_j-\sum_{m=1}^{4}\widetilde{w}_{g(j),m}\bar{z}_{jm} \tag{4}$$

式中 g(j)为目标 j 所属感官家族。各基学习器随后在完整外层训练集 T 上重拟合，对验证样本的最终连续预测由式(5)给出。

$$\hat{y}_{ij}=b_j+\sum_{m=1}^{4}\widetilde{w}_{g(j),m}f_m^{(T)}\left(x_i\right) \tag{5}$$

表 3 汇总 30 个外层训练折的权重，表示不同预测器在本数据中的组合方式，不作因果贡献比例解释。权重总和允许超过 1，自由截距同时校正中心偏移。

表 3 感官组收缩堆叠的平均非负权重

感官家族	ExtraTrees	HGB	Ridge	PLS2
香气	0.713	0.142	0.040	0.231
风味	0.784	0.146	0.033	0.202

感官家族	ExtraTrees	HGB	Ridge	PLS2
其他味觉	0.648	0.250	0.100	0.178
口感	0.643	0.207	0.172	0.219
总体喜好	0.642	0.180	0.046	0.192

0—6 相对强度映射

仅利用当前外层训练集中目标 j 的经验分布，将验证预测单调映射至 0—6 区间，定义为式(6)。其中 F 为训练目标的经验分布函数，指示函数统计不大于给定预测值的训练观测比例。

$$\hat{g}_{ij} = 6 \cdot \hat{F}_{j,tr}(\hat{y}_{ij}), \quad 0 \le \hat{g}_{ij} \le 6 \tag{6}$$

先逐折映射，再对 10 次重复的等级取平均，不能先汇总预测再用全体目标拟合映射。该等级保持维度内部的相对强弱次序，不能等同于原始评委平均分。附图 S1(b)展示固定训练折的映射曲线。

问题一求解

求解步骤与交叉验证

使用固定的 10 次重复、每次 3 折外层划分，尽量平衡细分风格；极小风格不强求每折均出现。每次重复中，每款啤酒恰好进入一个验证折。对每个外层训练集，依次进行内层 3 折参数选择、内层折外预测构造、共享及组内 NNLS 拟合、0.5 权重收缩与自由截距恢复，最后重拟合基学习器并预测外层验证集。随机种子以 20261023 为起点，并由重复、折及学习器索引确定。

对每个感官目标先单独计算指标，再对 50 个目标等权平均。式(7)、式(8)给出宏平均 R^2 与 MAE；n 为本次评价样本数，J=50，评价均值仅用于指标定义，不用于训练。RMSE 同样先按目标计算再平均。

$$R^2_{宏} = \frac{1}{J}\sum_{j=1}^{J}\left[1 - \frac{\sum_{i=1}^{n}\left(y_{ij} - \hat{y}_{ij}\right)^2}{\sum_{i=1}^{n}\left(y_{ij} - \bar{y}_j\right)^2}\right] \tag{7}$$

$$MAE_{宏} = \frac{1}{nJ}\sum_{j=1}^{J}\sum_{i=1}^{n}\left|y_{ij} - \hat{y}_{ij}\right| \tag{8}$$

Spearman 衡量连续值排序关系。七档分位代理 QWK 与相邻等级一致率分别将观测、预测按各自样本分位数分为七档，再计算一致性；它们是固定的事后排序诊断，不进入相对等级推断，也不是原始 0—6 评分准确率。以下主比较使用 10 个完整重复指标的均值与标准差；平均折外预测则先对每款啤酒的 10 次预测取平均，二者单列。

结果分析与模型对比

表 4 比较非线性基线及三种融合方法与最终模型，各模型使用相同外层划分，图 1 进一步展示重复间波动。最终模型提高解释度与连续预测精度，但在部分排序指标上没有取得最高值。

表 4 相同外层划分下的五项预测指标

方法	R^2（均值±SD）	MAE（均值±SD）	Spearman	代理 QWK	相邻等级一致率
ExtraTrees 基线	0.2359±0.0058	0.3422±0.0011	0.4144	0.4026	54.76%
连续凸融合	0.2482±0.0065	0.3397±0.0013	0.4273	0.4150	55.23%
连续—有序融合	0.2480±0.0060	0.3356±0.0014	0.4314	0.4194	55.52%
逐目标收缩融合	0.2472±0.0059	0.3360±0.0012	0.4295	0.4172	55.25%
感官组收缩 NNLS	0.2564±0.0070	0.3361±0.0016	0.4302	0.4183	55.42%

感官组收缩 NNLS 的重复均值 R^2 为 0.2564，MAE 为 0.3361，Spearman 为 0.4302，代理 QWK 为 0.4183，相邻等级一致率为 55.42%；五项均在 10 次重复中优于 ExtraTrees 基线。重复均值 RMSE 为 0.4714。连续—有序融合的 MAE 为 0.3356、Spearman 为 0.4314、代理 QWK 为 0.4194、相邻等级一致率为 55.52%，均略好于最终模型。选择最终模型依据连续解释度、平均折外精度及稳健性综合权衡，不将其表述为所有指标同时最优。

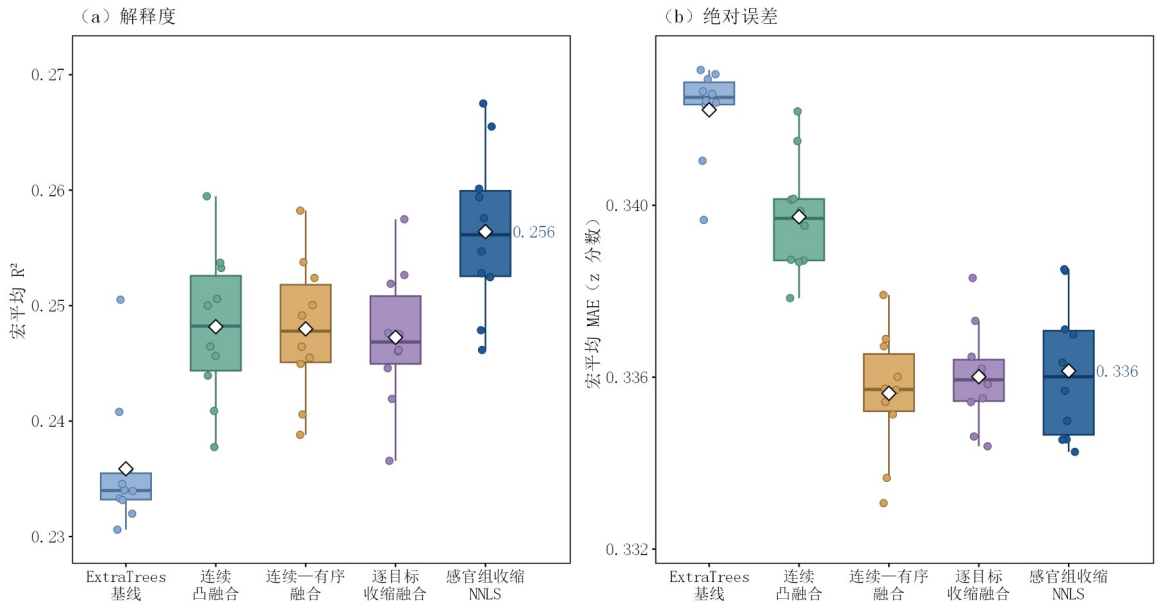


图 1 基线、融合与最终模型的重复折外性能比较（纵轴为局部范围）

平均折外预测的宏平均 R^2 为 0.2742、MAE 为 0.3315、RMSE 为 0.4655、Spearman 为 0.4487，代理 QWK 为 0.4368、相邻等级一致率为 56.42%。ExtraTrees 基线对应 R^2 为 0.2502、MAE 为 0.3389。平均预测降低了随机划分造成的波动，但这些指标不替代上文 10 次重复的均值；逐目标收缩融合的平均折外 Spearman 为 0.4511，仍高于最终模型。

图 2 按感官家族完整展示 50 个目标，避免只挑高分目标。目前 50 个目标的平均折外 R^2 为正。酸度、酒精感等信号较强，花香酯、陈旧酒花味及部分总体感受信号较弱。每个目标的解释度与秩相关不能互相替代，尤其在预测幅度收缩时，排序尚可仍可能伴随较低 R^2。

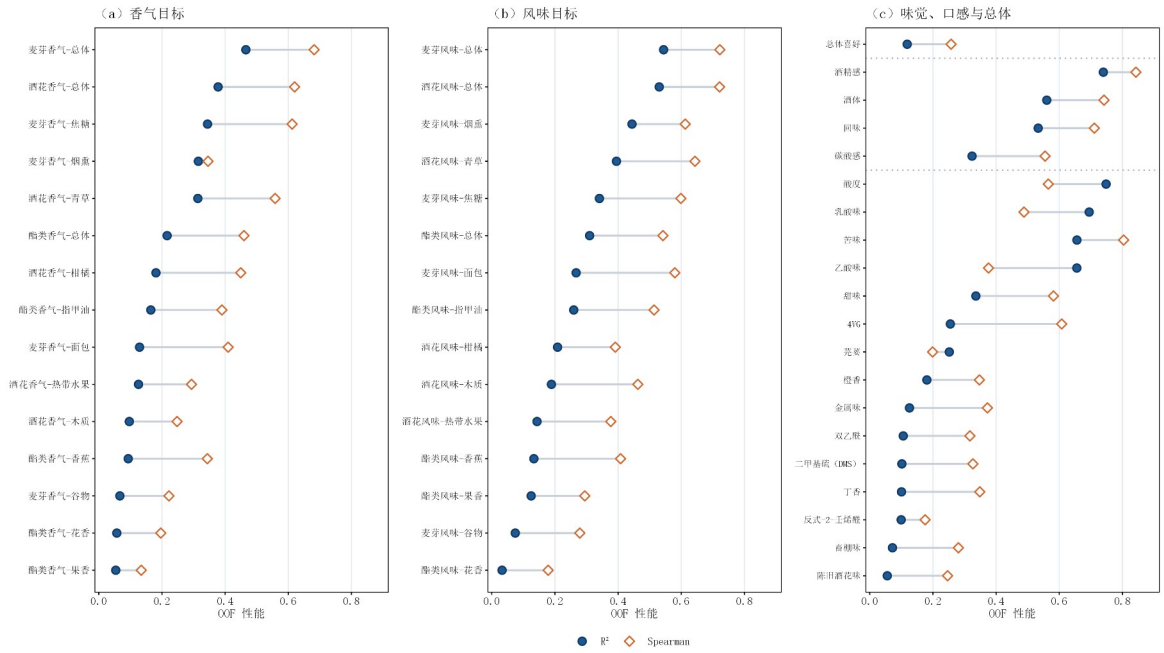


图 2 最终模型 50 个感官目标的平均折外 R^2 与 Spearman

图 3 沿用事先固定的酸度、4VG 和陈旧酒花味三个代表目标，分别检视较强、中等及较弱信号。虚线为理想预测，实线与阴影为观测—预测 OLS 拟合及均值置信带，不是单个新样本的预测区间。弱目标的预测仍向均值收缩，说明共享建模不能补足未测化学信息。

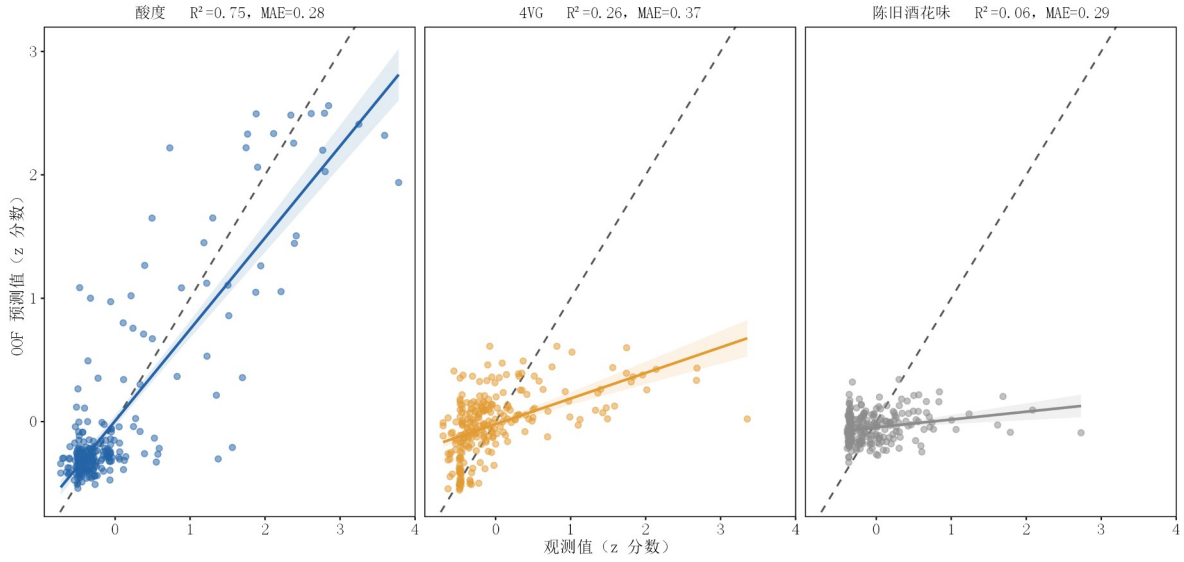


图 3 固定代表感官目标的平均折外观测—预测关系

模型检验与稳健性

输入中排除标识与风格标签，填补、秩变换、基学习器选择、堆叠与映射均在相应训练范围内进行。附件 2、5、6 的全量相关矩阵不用于本问筛选或调参。数据附带的标准化监督信号沿用官方定义；没有原始评委记录时，不能审计或消除其上游标准化过程可能带来的依赖。

为检验弱目标，以基线重复平均 R^2 预先固定最低的 13 个目标，另固定 10 个难目标，不能按最终模型重新挑选。尾部 13 目标的平均 R^2 从 0.0411 增至 0.0639，MAE 从 0.3277 降至 0.3227；10 个难目标平均 ΔR^2 为+0.0081、ΔMAE 为−0.0032。相较逐目标收缩融合尾部 MAE 仍略高，因此不声称弱目标普遍最优。图 4 同时展示风格误差和难目标的正负变化。

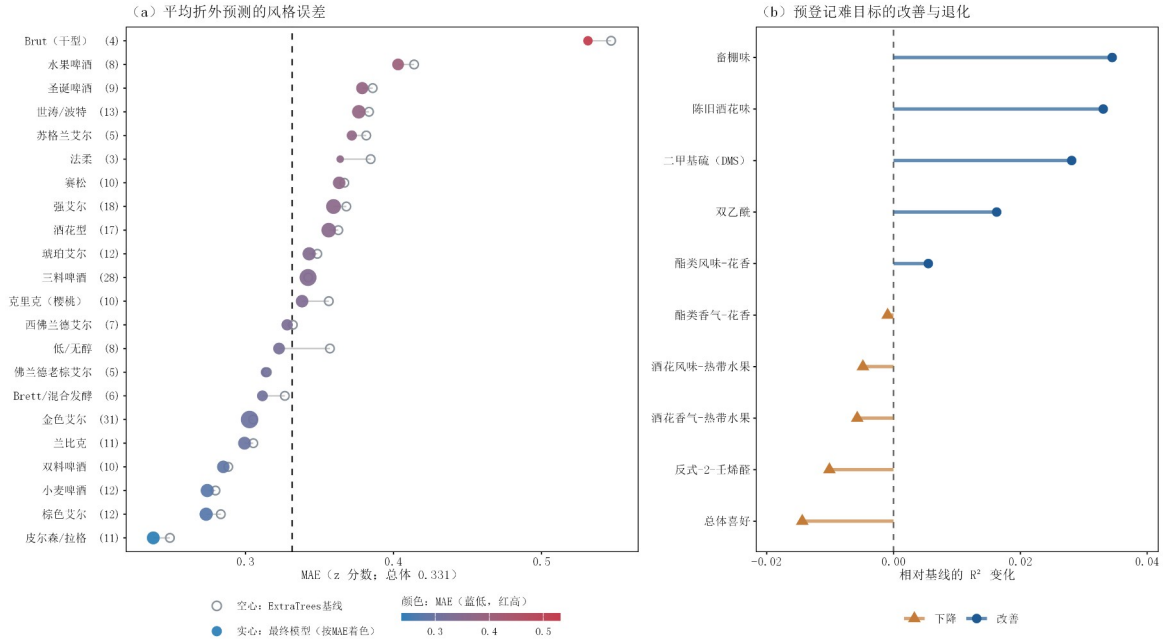


图 4 平均折外风格误差与预登记难目标的重复平均 R^2 变化

22 个风格中，20 个的重复平均 MAE 低于基线；最差风格 Brut 由 0.5512 降至 0.5348。佛兰德老棕艾尔与 Tripel 分别增加 0.0019 和 0.0011。总体喜好和反式-2-壬烯醛的 R^2 分别下降 0.0144 和 0.0101，MAE 分别增加 0.0042 和 0.0010；香气、风味的热带酒花维度也有小幅下降。风格内样本量较小，这些描述不构成未见风格的外推保证。

进一步在固定平均折外预测上，以啤酒为配对单位进行 20,000 次自助重采样。每款啤酒先对 50 个目标取绝对误差平均；秩误差则对各目标的观测和预测分别排名，以样本数减 1 归一化，再求绝对差。表 5 中的差值均为最终模型减基线，负值表示误差较低。

表 5 最终模型相对基线的配对自助重采样结果

指标	基线	最终模型	差值	95%百分位区间
MAE	0.338879	0.331498	-0.007381	[-0.009467, -0.005331]
归一化绝对秩误差	0.237023	0.233291	-0.003732	[-0.006055, -0.001502]

两个区间均位于 0 以下，支持当前固定预测下的平均改善。该自助区间不包含重训或模型选择的不确定性，10 次划分也共享样本，不作为 10 个独立研究处理。附图 S1(a)继续检查残差的极端区间偏差与方差变化。

表 6 比较预先设置的三种权重共享结构；共享权重反映全局共性，逐目标权重提供更细自由度，感官组结构处于二者之间。三种模型均固定 0.5 收缩，表内比较仅解释权重结构差异，不是对收缩系数的敏感性曲线。

表 6 权重共享结构消融

结构	R^2	MAE	Spearman	代理 QWK	相邻等级一致率
共享权重 NNLS	0.2551	0.3365	0.4298	0.4175	55.34%
感官组收缩 NNLS	0.2564	0.3361	0.4302	0.4183	55.42%
逐目标收缩 NNLS	0.2551	0.3364	0.4274	0.4157	55.43%

感官组收缩在此比较中取得较高解释度与较低 MAE；逐目标收缩的相邻等级一致率为 55.432%，略高于感官组的 55.423%，说明更细的权重并未带来各项指标一致改善。现有结果只支持固定结构下的比较；其他收缩强度、跨批次数据和原始 0—6 标签的检验仍待补充。

结果解释与模型评价

最终模型利用树模型、线性收缩与低秩多输出的误差互补，在严格外层验证中提高整体连续解释度，固定尾部和最差风格也优于基线。NNLS 权重保留可解释的非负组合，自由截距与组间收缩控制幅度和小组方差，适合作为后续感官画像的预测接口。

其限制有三点：少数难目标的化学信号仍不足，整体改善不能替代逐维可靠性；七档排序代理及相对强度映射均不能当作原始评委评分；重复验证、自助区间和标准化数据只支持当前商业啤酒样本范围，不能证明未见风格或合成化学谱上的真实感官安全。问题二的关联证据与问题三的候选设计仍须分别验证。

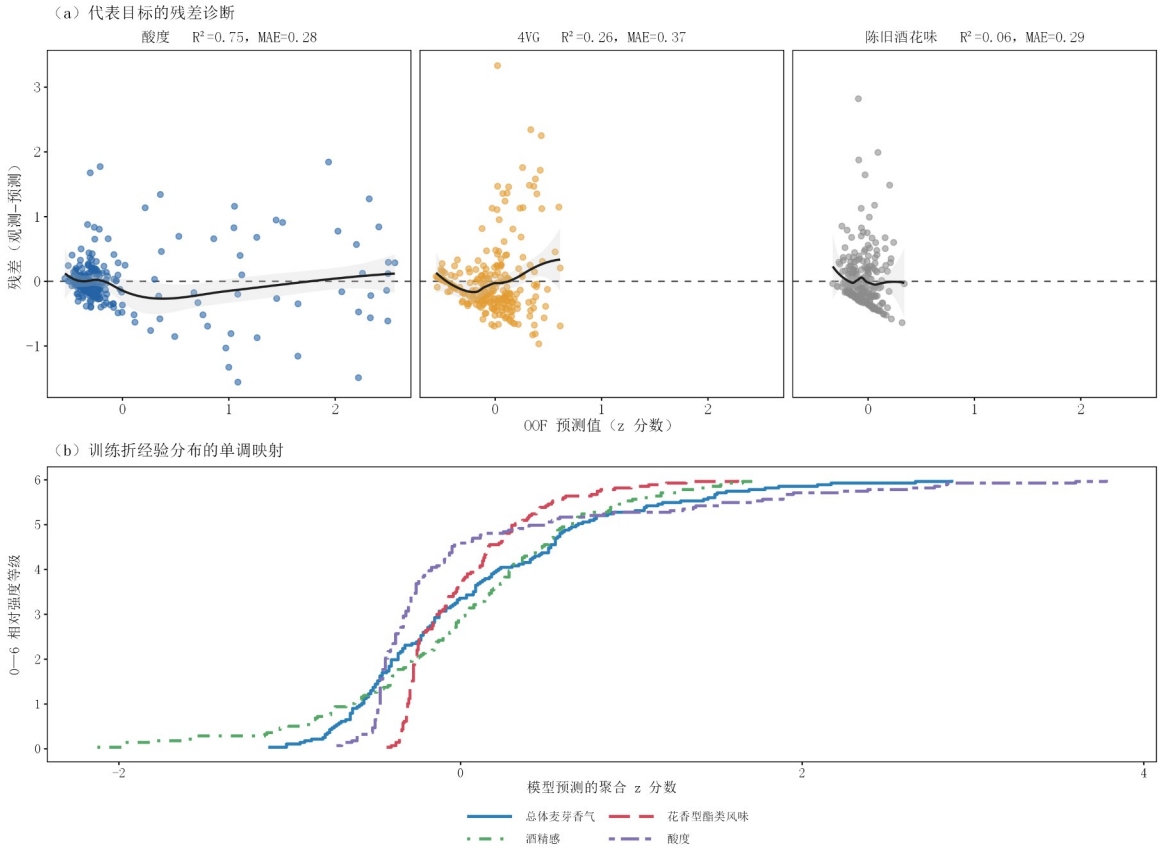
参考文献

[1] Breiman L. Stacked regressions. Machine Learning, 1996, 24: 49–64. DOI: 10.1023/A:1018046112532.

[2] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 2006, 63: 3–42. DOI: 10.1007/s10994-006-6226-1.

附图

附图 S1 合并残差诊断与训练折经验分布映射。残差图使用与图 3 相同的固定目标；映射示例仅使用首个重复的首个外层训练折，不使用验证观测拟合经验分布。程序及完整逐维表另附。



附图 S1 固定目标残差与训练折经验分布的相对量表映射

模型假设

为简化问题，本文做出以下假设：

假设 1：同一字段在样本之间具有可比的测量含义。附件 1 采用统一字段，附件 8 支持识别 37 个 log10 字段与 33 个原尺度字段，但字段尺度可识别不代表每个单元格一致。本文保留附件 1 给定值；若发现批次或单元格冲突，则单列核验，涉及绝对浓度的解释仅使用逐条确认的观测。

假设 2：在没有酒厂、批次及重复配方标识时，将每款啤酒作为一个观测单位。beer 字段完整且唯一，支持样本对齐；若后续获得同厂或同批信息，则改用分组交叉验证，并重新估计泛化误差。

假设 3：少量缺失不使整款样本失去预测价值，且缺失位置可能包含测量流程信息。输入共 73 个缺失单元，涉及 63 行 9 列，因此采用折内中位数填补并保留指示。若缺失机制明显依赖未观测浓度或系统故障，则补做删失模型或多重插补分析，降低有关预测的可信度。

假设 4：附件 4 中的连续值保留各目标内部的相对强弱顺序，可用于监督学习。附件 3 给出 0—6 评分定义，但没有完整的评委中心化、缩放参数，故不假设原始均分可逆恢复；若获得原始评分，改在原始量表建模并重新评价映射。

假设 5：当前样本覆盖范围内，不同感官家族可共享部分预测信息。五个家族和重复折内预测支持估计共享及组内权重，固定 0.5 收缩用于约束小组方差；若外部样本或未见风格出现结构漂移，则重新校准、分组验证或拒绝外推。

问题一分析

问题一要求由化学组成预测多维感官强度。本题可用样本为 250 款啤酒，输入包含 231 项化学与理化特征，监督信号为 50 个标准化连续感官属性。其困难在于输入维数接近样本量、化合物之间相关性较强，各感官家族的可预测性又不相同；附件中的标准化值还不能唯一还原为评委原始 0—6 分。风格分布不均，单一平均误差也可能掩盖缺陷味及小样本风格的失败。

本文按 beer 字段对齐附件，在训练折内完成填补与特征变换；以 ExtraTrees 为非线性基线，将 HGB、秩高斯 Ridge 和 PLS2 的互补预测纳入非负最小二乘堆叠，并通过感官组向共享权重收缩控制方差。采用 10 次重复、每次 3 折的外层验证，基学习器选择与堆叠权重估计均限制在外层训练数据内。评价同时覆盖解释度、绝对误差、排序一致性及弱目标、风格误差，0—6 仅按训练折经验分布给出相对强度。

附录 第一问源程序

运行环境为 Python 3.10 和 R。Python 依赖 numpy、pandas、scipy、scikit-learn；R 依赖 ggplot2、ragg、dplyr、tidyr、readr、patchwork、svglite。全部模型程序、固定划分和绘图数据随支撑材料提供，具体命令见程序目录 README。以下按文件名列出源程序；Word 中的自动换行不改变支撑材料文件。

baseline_models.py

```
"""Small, testable building blocks for the Q1 CPU baselines."""

from __future__ import annotations

from collections.abc import Callable, Sequence
from typing import Any

import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import MissingIndicator, SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.preprocessing import QuantileTransformer, StandardScaler
from sklearn.model_selection import KFold


def fit_predict_global_mean(y_train: np.ndarray, n_validation: int) -> np.ndarray:
    """Predict each target with its training-fold mean."""
    mean = np.asarray(y_train, dtype=float).mean(axis=0, keepdims=True)
    return np.repeat(mean, repeats=n_validation, axis=0)


def fit_predict_shrunk_style(
    y_train: np.ndarray,
    train_style: pd.Series,
    validation_style: pd.Series,
    shrinkage: float,
) -> np.ndarray:
    """Predict style means shrunk toward the training-fold global mean."""
    y_train = np.asarray(y_train, dtype=float)
    train_style = pd.Series(train_style).reset_index(drop=True)
    validation_style = pd.Series(validation_style).reset_index(drop=True)
    if len(y_train) != len(train_style):
        raise ValueError("y_train and train_style have different lengths")
    if shrinkage < 0:
        raise ValueError("shrinkage must be non-negative")

    global_mean = y_train.mean(axis=0)
    predictions = []
```

```
    for style in validation_style:
        mask = train_style.eq(style).to_numpy()
        if not mask.any():
            predictions.append(global_mean)
            continue
        n_style = int(mask.sum())
        style_mean = y_train[mask].mean(axis=0)
        prediction = (n_style * style_mean + shrinkage * global_mean) / (
            n_style + shrinkage
        )
        predictions.append(prediction)
    return np.asarray(predictions)


def fit_predict_ridge(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    alpha: float,
) -> np.ndarray:
    """Fit a fold-local impute/scale/Ridge pipeline and predict all targets."""
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=alpha)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def fit_predict_pls(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_components: int,
) -> np.ndarray:
    """Fit a fold-local impute/scale/PLS2 pipeline and predict all targets."""
    max_components = min(len(x_train) - 1, np.asarray(x_train).shape[1])
    if not 1 <= n_components <= max_components:
        raise ValueError(
            f"n_components={n_components} outside valid range 1..{max_components}"
        )
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            ("model", PLSRegression(n_components=n_components, scale=True, max_iter=1000)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def _quantile_preprocessor(n_train: int, random_state: int) -> FeatureUnion:
    return FeatureUnion(
        [
            (
                "continuous",
                Pipeline(
                    [
```

```
                        ("imputer", SimpleImputer(strategy="median")),
                        (
                            "quantile",
                            QuantileTransformer(
                                n_quantiles=min(100, n_train),
                                output_distribution="normal",
                                subsample=None,
                                random_state=random_state,
                            ),
                        ),
                    ]
                ),
            ),
            (
                "missing_indicator",
                MissingIndicator(features="missing-only", error_on_new=False),
            ),
        ]
    )


def fit_predict_ridge_quantile(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    alpha: float,
    random_state: int,
) -> np.ndarray:
    """Ridge after a training-fold rank-Gaussian transform."""
    pipeline = Pipeline(
        [
            ("preprocessor", _quantile_preprocessor(len(x_train), random_state)),
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=alpha)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def fit_predict_pls_quantile(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_components: int,
    random_state: int,
) -> np.ndarray:
    """PLS2 after a training-fold rank-Gaussian transform."""
    max_components = min(len(x_train) - 1, np.asarray(x_train).shape[1])
    if not 1 <= n_components <= max_components:
        raise ValueError(
            f"n_components={n_components} outside valid range 1..{max_components}"
        )
    pipeline = Pipeline(
        [
            ("preprocessor", _quantile_preprocessor(len(x_train), random_state)),
            ("model", PLSRegression(n_components=n_components, scale=True, max_iter=1000)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))
```

```
def fit_predict_extra_trees(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_estimators: int,
    max_features: float,
    min_samples_leaf: int,
    max_depth: int | None,
    random_state: int,
    n_jobs: int = -1,
) -> np.ndarray:
    """Fit a fold-local median-imputed multi-output ExtraTrees model."""
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            (
                "model",
                ExtraTreesRegressor(
                    n_estimators=n_estimators,
                    max_features=max_features,
                    min_samples_leaf=min_samples_leaf,
                    max_depth=max_depth,
                    random_state=random_state,
                    n_jobs=n_jobs,
                ),
            ),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def empirical_cdf_grade(
    reference_targets: np.ndarray, predicted_targets: np.ndarray
) -> np.ndarray:
    """Map predictions to [0, 6] using training-only empirical CDFs."""
    reference_targets = np.asarray(reference_targets, dtype=float)
    predicted_targets = np.asarray(predicted_targets, dtype=float)
    if reference_targets.ndim != 2 or predicted_targets.ndim != 2:
        raise ValueError("reference_targets and predicted_targets must be 2-D")
    if reference_targets.shape[1] != predicted_targets.shape[1]:
        raise ValueError("reference and prediction target counts disagree")
    grades = np.empty_like(predicted_targets, dtype=float)
    for target_position in range(reference_targets.shape[1]):
        ordered = np.sort(reference_targets[:, target_position])
        ranks = np.searchsorted(
            ordered, predicted_targets[:, target_position], side="right"
        )
        grades[:, target_position] = 6.0 * ranks / len(ordered)
    return grades


def select_parameter_by_mae(
    targets: np.ndarray,
    candidates: Sequence[Any],
    predict_fold: Callable[[np.ndarray, np.ndarray, Any], np.ndarray],
    n_splits: int,
    random_state: int,
) -> tuple[Any, dict[Any, float]]:
```

```
    """Select one candidate using mean inner-fold multi-output MAE."""
    if not candidates:
        raise ValueError("candidates must not be empty")
    targets = np.asarray(targets, dtype=float)
    splitter = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    scores: dict[Any, float] = {}
    for candidate in candidates:
        fold_scores = []
        for train_index, validation_index in splitter.split(np.arange(len(targets))):
            prediction = np.asarray(
                predict_fold(train_index, validation_index, candidate), dtype=float
            )
            fold_scores.append(
                float(np.mean(np.abs(targets[validation_index] - prediction)))
            )
        scores[candidate] = float(np.mean(fold_scores))

    selected = min(candidates, key=lambda candidate: scores[candidate])
    return selected, scores


def regression_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return unclipped regression metrics for one target."""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    if y_true.shape != y_pred.shape:
        raise ValueError("y_true and y_pred must have identical shapes")

    pred_sd = float(np.std(y_pred, ddof=0))
    if pred_sd == 0:
        spearman = np.nan
        calibration_intercept = float(np.mean(y_true))
        calibration_slope = 0.0
    else:
        spearman = float(pd.Series(y_true).corr(pd.Series(y_pred), method="spearman"))
        design = np.column_stack([np.ones(len(y_pred)), y_pred])
        calibration_intercept, calibration_slope = np.linalg.lstsq(
            design, y_true, rcond=None
        )[0]

    return {
        "r2": float(r2_score(y_true, y_pred)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "rmse": float(mean_squared_error(y_true, y_pred) ** 0.5),
        "spearman": spearman,
        "calibration_intercept": float(calibration_intercept),
        "calibration_slope": float(calibration_slope),
    }
```

ensemble_models.py

```
from __future__ import annotations
import numpy as np
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.impute import SimpleImputer

def fit_predict_hgb_multioutput(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
```

```
    *,
    max_leaf_nodes: int,
    min_samples_leaf: int,
    learning_rate: float,
    l2_regularization: float,
    max_iter: int,
    random_state: int,
) -> np.ndarray:
    """Fit one fold-local HistGradientBoostingRegressor per target."""
    targets = np.asarray(y_train, dtype=float)
    if targets.ndim != 2 or not np.isfinite(targets).all():
        raise ValueError("y_train must be a finite two-dimensional matrix")
    if len(x_train) != len(targets):
        raise ValueError("x_train and y_train have different lengths")
    imputer = SimpleImputer(strategy="median", add_indicator=True)
    train_features = imputer.fit_transform(np.asarray(x_train, dtype=float))
    validation_features = imputer.transform(np.asarray(x_validation, dtype=float))
    columns = []
    for target_position in range(targets.shape[1]):
        model = HistGradientBoostingRegressor(
            max_leaf_nodes=max_leaf_nodes,
            min_samples_leaf=min_samples_leaf,
            learning_rate=learning_rate,
            l2_regularization=l2_regularization,
            max_iter=max_iter,
            random_state=random_state + target_position,
        )
        model.fit(train_features, targets[:, target_position])
        columns.append(model.predict(validation_features))
    prediction = np.column_stack(columns)
    if not np.isfinite(prediction).all():
        raise RuntimeError("continuous HGB produced non-finite predictions")
    return prediction
```

model_config.json

```
{
  "base_models": [
    "extra_trees",
    "hgb",
    "ridge_quantile",
    "pls2_quantile"
  ],
  "extra_trees": {
    "n_estimators": 300,
    "max_features": [
      0.33,
      0.6,
      1.0
    ],
    "min_samples_leaf": [
      1,
      2,
      4,
      8
    ],
    "max_depth": null
  },
  "hgb_profiles": [
    {
      "name": "compact",
```

```
      "max_leaf_nodes": 7,
      "min_samples_leaf": 20,
      "learning_rate": 0.05,
      "l2_regularization": 1.0,
      "max_iter": 100
    },
    {
      "name": "balanced",
      "max_leaf_nodes": 15,
      "min_samples_leaf": 20,
      "learning_rate": 0.05,
      "l2_regularization": 1.0,
      "max_iter": 100
    }
  ],
  "ridge_alphas": [
    0.1,
    1.0,
    10.0,
    100.0,
    1000.0
  ],
  "pls_components": [
    2,
    5,
    10,
    15,
    20
  ]
}
```

nnls_stacking.py

```
"""Nonnegative OOF model weights with free target intercepts and fixed shrinkage."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import nnls


@dataclass(frozen=True)
class NNLSStackingFit:
    weights: np.ndarray
    intercept: np.ndarray
    global_weights: np.ndarray
    scope: str
    shrinkage: float

    def predict(self, stack) -> np.ndarray:
        values = np.asarray(stack, dtype=float)
        if values.ndim != 3 or values.shape[1:] != self.weights.shape:
            raise ValueError("prediction tensor must have the fitted targets/models shape")
        if not np.isfinite(values).all():
            raise ValueError("prediction tensor must be finite")
        return np.einsum("ntm,tm->nt", values, self.weights) + self.intercept


def _solve(centered_stack, centered_y, scales) -> np.ndarray:
```

```
    design = (centered_stack / scales[None, :, None]).reshape(-1, centered_stack.shape[2])
    response = (centered_y / scales[None, :]).reshape(-1)
    if not np.any(design):
        return np.zeros(design.shape[1], dtype=float)
    weights, _ = nnls(design, response, maxiter=max(100, design.shape[1] * 30))
    return weights


def fit_nnls_stacking(stack, y, *, groups=None, scope="shared", shrinkage=.5) -> NNLSStackingFit:
    """Fit on outer-training inner-OOF predictions and their training labels only.

    Weights are nonnegative but need not sum to one. Target centering removes
    intercepts before NNLS, so an intercept is free to be negative afterward.
    Standardizing each training target gives shared/group fits equal target units.
    """
    predictions, truth = np.asarray(stack, dtype=float), np.asarray(y, dtype=float)
    if (predictions.ndim != 3 or min(predictions.shape) < 1
            or predictions.shape[0] < 3 or truth.shape != predictions.shape[:2]
            or not np.isfinite(predictions).all() or not np.isfinite(truth).all()):
        raise ValueError("finite samples-by-targets-by-models predictions must align with y")
    if scope not in {"shared", "group", "target"}:
        raise ValueError("scope must be shared, group or target")
    if not np.isfinite(shrinkage) or not 0 <= shrinkage <= 1:
        raise ValueError("shrinkage must be in [0, 1]")
    if scope == "group" and (groups is None or len(groups) != truth.shape[1]):
        raise ValueError("group labels must cover all registered targets")
    means = predictions.mean(axis=0)
    target_means = truth.mean(axis=0)
    centered, y_centered = predictions - means, truth - target_means
    scales = np.std(truth, axis=0)
    scales = np.where(scales > 1e-12, scales, 1.)
    global_weights = _solve(centered, y_centered, scales)
    weights = np.tile(global_weights, (truth.shape[1], 1))
    if scope != "shared":
        labels = np.arange(truth.shape[1]) if scope == "target" else np.asarray(groups, dtype=str)
        for label in dict.fromkeys(labels.tolist()):
            positions = np.flatnonzero(labels == label)
            local = _solve(centered[:, positions], y_centered[:, positions], scales[positions])
            weights[positions] = (1. - shrinkage) * local + shrinkage * global_weights
    intercept = target_means - np.einsum("tm,tm->t", means, weights)
    return NNLSStackingFit(weights.copy(), intercept.copy(), global_weights.copy(),
                           str(scope), 1. if scope == "shared" else float(shrinkage))
```

plot_final.R

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(ggplot2)
  library(dplyr)
  library(tidyr)
  library(readr)
  library(patchwork)
})

source("plot_style.R")

data_dir <- "../数据与结果"
figure_dir <- Sys.getenv(
  "Q1_FIGURE_DIR",
  unset = "../图片"
```

```
)
dir.create(figure_dir, recursive = TRUE, showWarnings = FALSE)
save_png <- save_paper
save_paper <- function(plot, file, width = 6, height = 4, dpi = 300) {
  save_png(plot, file, width, height, dpi)
  ggsave(sub("\\.png$", ".svg", file), plot, device = svglite::svglite,
         width = width, height = height, units = "in", bg = "white")
}

# 论文图文件内不写整图标题。图题只保存在 Q1_图清单.csv，
# 后续由 DOCX 在图下方统一排版。面板标识可用 (a)(b)，分面标签无底色并加粗。

group_labels <- c(
  "Aroma" = "香气",
  "Flavour-core" = "风味主体",
  "Taste-other" = "其他味觉",
  "Palate" = "口感",
  "Overall" = "总体"
)
group_colors <- c(
  "Aroma" = "#3B75AF",
  "Flavour-core" = "#EF8636",
  "Taste-other" = "#55A868",
  "Palate" = "#C44E52",
  "Overall" = "#8172B3"
)
tier_colors <- c("高" = "#2563A6", "中" = "#E39C34", "低" = "#8C8C8C")

# 原始字段代码继续留在数据层；论文图的刻度与图例使用中文展示名。
target_labels_zh <- c(
  "A_malt_all" = "麦芽香气-总体",
  "A_malt_grain" = "麦芽香气-谷物",
  "A_malt_bread" = "麦芽香气-面包",
  "A_malt_cara" = "麦芽香气-焦糖",
  "A_malt_burn" = "麦芽香气-烟熏",
  "A_hops_all" = "酒花香气-总体",
  "A_hops_citrus" = "酒花香气-柑橘",
  "A_hops_tropical" = "酒花香气-热带水果",
  "A_hops_noble" = "酒花香气-青草",
  "A_hops_woody" = "酒花香气-木质",
  "A_esters_all" = "酯类香气-总体",
  "A_esters_ethac" = "酯类香气-指甲油",
  "A_esters_isoaa" = "酯类香气-香蕉",
  "A_esters_flower" = "酯类香气-花香",
  "A_esters_fruity" = "酯类香气-果香",
  "F_malt_all" = "麦芽风味-总体",
  "F_malt_grain" = "麦芽风味-谷物",
  "F_malt_bread" = "麦芽风味-面包",
  "F_malt_cara" = "麦芽风味-焦糖",
  "F_malt_burn" = "麦芽风味-烟熏",
  "F_hops_all" = "酒花风味-总体",
  "F_hops_citrus" = "酒花风味-柑橘",
  "F_hops_tropical" = "酒花风味-热带水果",
  "F_hops_noble" = "酒花风味-青草",
  "F_hops_woody" = "酒花风味-木质",
  "F_esters_all" = "酯类风味-总体",
  "F_esters_ethac" = "酯类风味-指甲油",
  "F_esters_isoaa" = "酯类风味-香蕉",
  "F_esters_flower" = "酯类风味-花香",
  "F_esters_fruity" = "酯类风味-果香",
  "acidity" = "酸度",
```

```
  "bitternes" = "苦味",
  "sweetness" = "甜味",
  "X4vg" = "4VG",
  "diacetyl" = "双乙酰",
  "dms" = "二甲基硫（DMS）",
  "metallic" = "金属味",
  "stale_hops" = "陈旧酒花味",
  "t2n" = "反式-2-壬烯醛",
  "orange" = "橙香",
  "coriander" = "芫荽",
  "clove" = "丁香",
  "lactic" = "乳酸味",
  "acetic" = "乙酸味",
  "barnyard" = "畜棚味",
  "alcohol" = "酒精感",
  "aftertaste" = "回味",
  "body" = "酒体",
  "co2" = "碳酸感",
  "overall" = "总体喜好"
)

style_labels_zh <- c(
  "Amber" = "琥珀艾尔", "Blond" = "金色艾尔",
  "Brett/cofermented" = "Brett/混合发酵", "Brown" = "棕色艾尔",
  "Brut" = "Brut（干型）", "Christmas" = "圣诞啤酒",
  "Dubbel" = "双料啤酒", "Faro" = "法柔",
  "Flanders old brown" = "佛兰德老棕艾尔", "Fruitbeer" = "水果啤酒",
  "Hoppy" = "酒花型", "Kriek" = "克里克（樱桃）",
  "Lambic" = "兰比克", "Low/No alcohol" = "低/无醇",
  "Pils/Lager" = "皮尔森/拉格", "Saison" = "赛松",
  "Scotch" = "苏格兰艾尔", "Stout/Porter" = "世涛/波特",
  "Strong ale" = "强艾尔", "Tripel" = "三料啤酒",
  "West Flanders ale" = "西佛兰德艾尔", "Wheat" = "小麦啤酒"
)

panel_header_theme <- theme(
  plot.subtitle = element_text(
    colour = "black", face = "bold", size = 9.5, hjust = 0,
    margin = margin(b = 5)
  )
)

# 图 1：相同重复划分下比较基线、三种融合方法和最终堆叠模型。
comparison <- read_csv(file.path(data_dir, "model_comparison.csv"),
                       show_col_types = FALSE)
model_levels <- c("ExtraTrees 基线", "连续凸融合", "连续—有序融合",
                  "逐目标收缩融合", "感官组收缩 NNLS")
model_colors <- c(
  "ExtraTrees 基线" = "#88A9D3",
  "连续凸融合" = "#67A995",
  "连续—有序融合" = "#D4A35B",
  "逐目标收缩融合" = "#9A82B8",
  "感官组收缩 NNLS" = "#1F5A94"
)
model_outlines <- c(
  "ExtraTrees 基线" = "#4F719B",
  "连续凸融合" = "#3F7568",
  "连续—有序融合" = "#8C6735",
  "逐目标收缩融合" = "#66527F",
  "感官组收缩 NNLS" = "#123A65"
)
```

```
comparison <- comparison %>%
  mutate(display_name = factor(display_name, levels = model_levels))
focus_summary <- comparison %>%
  filter(display_name == "感官组收缩NNLS") %>%
  summarise(
    r2 = mean(macro_r2),
    mae = mean(macro_mae),
    r2_label_y = mean(macro_r2),
    mae_label_y = mean(macro_mae)
  ) %>%
  mutate(display_name = factor("感官组收缩NNLS", levels = model_levels))

p1a <- ggplot(comparison, aes(display_name, macro_r2,
                              fill = display_name, colour = display_name)) +
  geom_boxplot(width = 0.56, linewidth = 0.38, outlier.shape = NA, alpha = 0.88) +
  geom_point(position = position_jitter(width = 0.10, height = 0,
                                        seed = 20261023),
             shape = 21, size = 1.55, stroke = 0.28) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.35,
               stroke = 0.45, fill = "white", colour = "black") +
  geom_text(
    data = focus_summary,
    aes(x = display_name, y = r2_label_y, label = sprintf("%.3f", r2)),
    inherit.aes = FALSE, colour = "#123A65", size = 3.0,
    fontface = "bold", vjust = 0.5, hjust = 0, nudge_x = 0.30
  ) +
  scale_fill_manual(values = model_colors) +
  scale_colour_manual(values = model_outlines) +
  scale_y_continuous(expand = expansion(mult = c(0.08, 0.16))) +
  scale_x_discrete(labels = c("ExtraTrees\n基线", "连续\n凸融合", "连续—有序\n融合", "逐目标\n收缩融合", "感
官组收缩\nNNLS"), expand = expansion(add = c(.6,.95))) +
  labs(x = NULL, y = "宏平均 R²", subtitle = "（a）解释度") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5, size = 8))

p1b <- ggplot(comparison, aes(display_name, macro_mae,
                              fill = display_name, colour = display_name)) +
  geom_boxplot(width = 0.56, linewidth = 0.38, outlier.shape = NA, alpha = 0.88) +
  geom_point(position = position_jitter(width = 0.10, height = 0,
                                        seed = 20261023),
             shape = 21, size = 1.55, stroke = 0.28) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.35,
               stroke = 0.45, fill = "white", colour = "black") +
  geom_text(
    data = focus_summary,
    aes(x = display_name, y = mae_label_y, label = sprintf("%.3f", mae)),
    inherit.aes = FALSE, colour = "#123A65", size = 3.0,
    fontface = "bold", vjust = 0.5, hjust = 0, nudge_x = 0.30
  ) +
  scale_fill_manual(values = model_colors) +
  scale_colour_manual(values = model_outlines) +
  scale_y_continuous(expand = expansion(mult = c(0.14, 0.08))) +
  scale_x_discrete(labels = c("ExtraTrees\n基线", "连续\n凸融合", "连续—有序\n融合", "逐目标\n收缩融合", "感
官组收缩\nNNLS"), expand = expansion(add = c(.6,.95))) +
  labs(x = NULL, y = "宏平均 MAE（z 分数）", subtitle = "（b）绝对误差") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5, size = 8))

p1 <- (p1a | p1b)
```

```
save_paper(p1, file.path(figure_dir, "图1_模型对比.png"),
           width = 8.2, height = 4.35)

# 图2：逐目标 OOF 性能图谱。三栏分别承载香气、风味和其余目标，
# 每行用同一条短连线比较 R² 与 Spearman，避免 50 行双栏图的视觉拥挤。
target_metrics <- read_csv(file.path(data_dir, "target_metrics_final.csv"),
                           show_col_types = FALSE)
target_metrics <- target_metrics %>%
  mutate(
    panel_group = case_when(
      target_group == "Aroma" ~ "香气",
      target_group == "Flavour-core" ~ "风味",
      TRUE ~ "味觉·口感·总体"
    ),
    detail_order = case_when(
      target_group == "Taste-other" ~ 1L,
      target_group == "Palate" ~ 2L,
      target_group == "Overall" ~ 3L,
      TRUE ~ 1L
    )
  )

target_panel_plot <- function(data, panel_title, y_text_size = 6.5,
                              separators = numeric()) {
  panel_data <- data %>%
    arrange(detail_order, r2) %>%
    mutate(
      target_label = unname(target_labels_zh[target_code]),
      target_label = factor(target_label, levels = target_label)
    )
  metric_long <- bind_rows(
    panel_data %>% transmute(target_label, value = r2, metric = "R²"),
    panel_data %>% transmute(target_label, value = spearman,
                             metric = "Spearman")
  )

  plot <- ggplot(panel_data, aes(y = target_label)) +
    geom_vline(xintercept = c(0.2, 0.4, 0.6, 0.8),
               colour = "#E5E9EF", linewidth = 0.34) +
    geom_segment(aes(x = r2, xend = spearman, yend = target_label),
                 colour = "#C9D0DA", linewidth = 0.62, lineend = "round") +
    geom_point(data = metric_long,
               aes(x = value, shape = metric, fill = metric, colour = metric),
               size = 2.05, stroke = 0.72) +
    scale_shape_manual(values = c("R²" = 21, "Spearman" = 23)) +
    scale_fill_manual(values = c("R²" = "#1F5A94", "Spearman" = "white")) +
    scale_colour_manual(values = c("R²" = "#123A65",
                                   "Spearman" = "#D06B32")) +
    scale_x_continuous(limits = c(0, 0.9), breaks = seq(0, 0.8, 0.2),
                       expand = expansion(mult = c(0.015, 0.02))) +
    labs(x = "OOF 性能", y = NULL, subtitle = panel_title) +
    theme_mcup(base_size = 8.6, legend = "right") +
    panel_header_theme +
    theme(
      axis.text.y = element_text(size = y_text_size),
      legend.position = "bottom",
      legend.direction = "horizontal",
      legend.text = element_text(size = 7.2),
      plot.margin = margin(4, 5, 3, 4)
    )
```

```
  if (length(separators) > 0) {
    plot <- plot + geom_hline(
      yintercept = separators, colour = "#9AA3AE",
      linewidth = 0.42, linetype = "dotted"
    )
  }
  plot
}

p2a <- target_panel_plot(
  filter(target_metrics, panel_group == "香气"),
  "（a）香气目标"
)
p2b <- target_panel_plot(
  filter(target_metrics, panel_group == "风味"),
  "（b）风味目标"
)
p2c <- target_panel_plot(
  filter(target_metrics, panel_group == "味觉·口感·总体"),
  "（c）味觉、口感与总体", y_text_size = 6.0,
  separators = c(15.5, 19.5)
)

p2 <- (p2a | p2b | p2c) +
  plot_layout(widths = c(1, 1, 1), guides = "collect") &
  theme(legend.position = "bottom")
save_paper(p2, file.path(figure_dir, "图2_逐目标预测性能.png"),
           width = 11.2, height = 6.3)

# 图3：高/中/低各保留一个预登记代表目标，减少重复子图。
representative <- read_csv(file.path(data_dir, "representative_oof.csv.gz"),
                           show_col_types = FALSE)
representative3 <- representative %>%
  filter(target_code %in% c("acidity", "X4vg", "stale_hops")) %>%
  left_join(target_metrics %>% select(target_code, r2, mae), by = "target_code") %>%
  mutate(
    facet_label_clean = paste0(unname(target_labels_zh[target_code]),
                               "   R²=", sprintf("%.2f", r2),
                               "，MAE=", sprintf("%.2f", mae)),
    facet_label_clean = factor(
      facet_label_clean,
      levels = unique(facet_label_clean[order(match(target_code, c("acidity", "X4vg", "stale_hops")))])
    )
  )

p3 <- ggplot(representative3, aes(y_true, y_pred_mean)) +
  geom_abline(slope = 1, intercept = 0, linetype = 2, colour = "#555555",
              linewidth = 0.48) +
  geom_smooth(aes(colour = tier, fill = tier), method = "lm", formula = y ~ x,
              se = TRUE, linewidth = 0.68, alpha = 0.12) +
  geom_point(aes(colour = tier), size = 1.25, alpha = 0.52) +
  facet_wrap(~facet_label_clean, nrow = 1) +
  scale_colour_manual(values = tier_colors) +
  scale_fill_manual(values = tier_colors) +
  labs(x = "观测值（z 分数）", y = "OOF 预测值（z 分数）") +
  theme_mcup(base_size = 9.5, legend = "none") +
  theme(strip.background = element_blank(),
        strip.text = element_text(face = "bold", size = 8.5,
                                  margin = margin(1, 0, 4, 0)))
save_paper(p3, file.path(figure_dir, "图3_OOF观测预测.png"),
           width = 8.8, height = 4.2)
```

```
# 图 4：将风格稳健性与 0–6 输出口径组成横版双联图。
style_metrics <- read_csv(file.path(data_dir, "style_metrics_final.csv"),
                          show_col_types = FALSE) %>%
  arrange(mae_all_targets) %>%
  mutate(tasting_category_fine = factor(tasting_category_fine,
                                        levels = tasting_category_fine))
style_axis_labels <- setNames(
  paste0(unname(style_labels_zh[as.character(style_metrics$tasting_category_fine)]),
         "  (", style_metrics$n_beers, ")"),
  as.character(style_metrics$tasting_category_fine)
)
mean_oof_mae <- mean(target_metrics$mae)
p4a <- ggplot(style_metrics,
              aes(mae_all_targets, tasting_category_fine,
                  colour = mae_all_targets)) +
  geom_vline(xintercept = mean_oof_mae, linetype = 2,
             colour = "#222222", linewidth = 0.48) +
  geom_segment(aes(x = baseline_mae, xend = mae_all_targets,
                   yend = tasting_category_fine), colour = "grey78", linewidth = 0.45) +
  geom_point(aes(x = baseline_mae, shape = "ExtraTrees 基线"),
             colour = "#8A929C", size = 2.1, stroke = 0.7) +
  geom_point(aes(size = n_beers, shape = "最终模型"), alpha = 0.93) +
  scale_shape_manual(values = c("ExtraTrees 基线" = 1, "最终模型" = 16),
                     breaks = c("ExtraTrees 基线", "最终模型"),
                     labels = c("空心：ExtraTrees 基线", "实心：最终模型（按 MAE 着色）"),
                     name = NULL) +
  guides(shape = guide_legend(ncol = 1, override.aes = list(
    colour = c("#8A929C", "#2C7FB8"), size = c(2.3, 2.8)))) +
  scale_colour_gradient(
    low = "#2C7FB8", high = "#C43C4E",
    name = "颜色：MAE（蓝低，红高）",
    guide = guide_colourbar(
      direction = "horizontal", title.position = "top",
      barwidth = unit(3.6, "cm"), barheight = unit(0.18, "cm")
    )
  ) +
  scale_size_continuous(range = c(2.0, 5.0), guide = "none") +
  scale_y_discrete(labels = style_axis_labels) +
  scale_x_continuous(expand = expansion(mult = c(0.06, 0.06))) +
  labs(x = sprintf("MAE（z 分数；总体 %.3f）", mean_oof_mae), y = NULL,
       subtitle = "（a）平均折外预测的风格误差") +
  theme_mcup(base_size = 8.8, legend = "none") +
  panel_header_theme +
  theme(axis.text.y = element_text(size = 7.2),
        legend.position = "bottom",
        legend.direction = "horizontal",
        legend.title = element_text(face = "bold", size = 7.3),
        legend.text = element_text(size = 6.8))

mapping <- read_csv(file.path(data_dir, "scale_mapping_example.csv"),
                    show_col_types = FALSE) %>%
  mutate(
    target_display = recode(
      target_code,
      "A_malt_all" = "总体麦芽香气",
      "F_esters_flower" = "花香型酯类风味",
      "acidity" = "酸度",
      "alcohol" = "酒精感"
    )
  )
```

```
mapping_colors <- c(
  "总体麦芽香气" = "#2C7FB8",
  "花香型酯类风味" = "#D1495B",
  "酒精感" = "#55A868",
  "酸度" = "#8172B3"
)
mapping_linetypes <- c(
  "总体麦芽香气" = "solid",
  "花香型酯类风味" = "longdash",
  "酒精感" = "dotdash",
  "酸度" = "twodash"
)
p4b <- ggplot(mapping,
              aes(z_score, grade_0_6,
                  colour = target_display, linetype = target_display)) +
  geom_step(linewidth = 0.82, direction = "hv") +
  scale_colour_manual(values = mapping_colors, name = NULL) +
  scale_linetype_manual(values = mapping_linetypes, name = NULL) +
  scale_y_continuous(breaks = 0:6, limits = c(0, 6)) +
  labs(x = "模型预测的聚合 z 分数", y = "0–6 相对强度等级",
       subtitle = "（b）训练折经验分布的单调映射") +
  guides(
    colour = guide_legend(nrow = 2, byrow = TRUE),
    linetype = guide_legend(nrow = 2, byrow = TRUE)
  ) +
  theme_mcup(base_size = 8.8, legend = "right") +
  panel_header_theme +
  theme(
    legend.position = "bottom",
    legend.direction = "horizontal",
    legend.text = element_text(size = 6.8),
    legend.key.width = unit(1.15, "cm"),
    legend.spacing.x = unit(0.12, "cm")
  )

hard <- read_csv(file.path(data_dir, "preregistered_hard_targets.csv"),
                 show_col_types = FALSE) %>%
  arrange(delta_r2) %>%
  mutate(target_label = factor(unname(target_labels_zh[target_code]),
                               levels = unname(target_labels_zh[target_code])),
         change = ifelse(delta_r2 >= 0, "改善", "下降"))
p4hard <- ggplot(hard, aes(delta_r2, target_label, colour = change)) +
  geom_vline(xintercept = 0, linetype = 2, linewidth = 0.45, colour = "#555555") +
  geom_segment(aes(x = 0, xend = delta_r2, yend = target_label),
               linewidth = 1.0, alpha = .7) +
  geom_point(aes(shape = change), size = 2.6) +
  scale_colour_manual(values = c("改善" = "#1F5A94", "下降" = "#C57C39"), name = NULL) +
  scale_shape_manual(values = c("改善" = 16, "下降" = 17), name = NULL) +
  scale_x_continuous(expand = expansion(mult = c(.15,.15))) +
  labs(x = "相对基线的 R² 变化", y = NULL, subtitle = "（b）预登记难目标的改善与退化") +
  theme_mcup(base_size = 8.8, legend = "right") + panel_header_theme +
  theme(legend.position = "bottom", axis.text.y = element_text(size = 7.5))
p4 <- (p4a | p4hard) +
  plot_layout(widths = c(1.12, 0.88))
save_paper(p4, file.path(figure_dir, "图4_风格与难目标稳健性.png"),
           width = 10.5, height = 5.9)

# 附图 S1：保留诊断价值，但不占用正文图数。
p_s1 <- ggplot(representative3, aes(y_pred_mean, residual)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "#555555", linewidth = 0.45) +
  geom_point(aes(colour = tier), size = 1.30, alpha = 0.58) +
```

```
  geom_smooth(method = "loess", formula = y ~ x, se = TRUE,
              colour = "#1F1F1F", fill = "#C9C9C9", linewidth = 0.62,
              alpha = 0.24) +
  facet_wrap(~facet_label_clean, nrow = 1) +
  scale_colour_manual(values = tier_colors) +
  labs(x = "OOF 预测值（z 分数）", y = "残差（观测−预测）", subtitle = "（a）代表目标的残差诊断") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme + theme(strip.background = element_blank(),
        strip.text = element_text(face = "bold", size = 8.5,
                                  margin = margin(1, 0, 4, 0)))
p_s1_combined <- p_s1 / p4b + plot_layout(heights = c(1, 0.85))
save_paper(p_s1_combined, file.path(figure_dir, "附图 S1_残差诊断与量表映射.png"),
           width = 9.5, height = 7.0)

manifest <- tibble::tribble(
  ~figure_no, ~role, ~path, ~caption, ~explanation,
  "1", "main", file.path(figure_dir, "图 1_模型对比.png"),
  "图 1 基线、融合与感官组收缩 NNLS 的重复折外性能比较",
  "散点表示 10 次重复，箱体表示重复间分布；两个面板分别衡量解释度和绝对误差，纵轴为局部范围。",
  "2", "main", file.path(figure_dir, "图 2_逐目标预测性能.png"),
  "图 2 最终模型 50 个感官目标的 OOF R² 与 Spearman 性能图谱",
  "三栏按感官家族展示全部目标；同一目标的 R² 与 Spearman 共同刻画可预测性异质性。",
  "3", "main", file.path(figure_dir, "图 3_OOF 观测预测.png"),
  "图 3 高、中、低可预测感官目标的 OOF 观测—预测关系",
  "酸度、4VG 和陈旧酒花味为此前固定的代表目标；虚线为理想预测，实线及阴影为 OLS 拟合及其均值置信带。",
  "4", "main", file.path(figure_dir, "图 4_风格与难目标稳健性.png"),
  "图 4 最终模型的风格误差及预登记难目标的稳健性检验",
  "左图为 mean-OOF 风格误差，空心点为基线、实心点为最终模型，括号和点大小表示样本量；右图为 10 次重复平均 R² 差值。",
  "S1", "supplementary", file.path(figure_dir, "附图 S1_残差诊断与量表映射.png"),
  "附图 S1 折外残差诊断与训练折经验分布的相对量表映射",
  "上图 LOESS 用于检查系统偏差；下图为固定训练折的 CDF 示例，0–6 仅表示相对强度，图例置于下方。"
)
write_csv(manifest, file.path(figure_dir, "Q1_图清单.csv"))
cat("Generated 4 main figures and 1 supplementary figure at", figure_dir, "\n")
```

plot_style.R

```
# 论文出图统一样式（R / ggplot2）
#
# 视觉规范来源：40 篇 M 杯优秀论文 615 张图表实测 + 逐页目检（docs/M 杯优秀论文特点、题型特点.md §4.5）
#   - 白底、黑色细边框、无网格、图例内嵌且无边框、3:2 横版
#   - 中文宋体 + 拉丁 Times 风格（SimSun 自带）
#   - 训练曲线四色；PR 总曲线深蓝加粗；混淆矩阵 Blues；热力图 hot
# 项目强制规则：图片文件内不放整图标题/图号，图题在论文中置于图下；
# 分面标签无灰色底框并加粗，轴标题加粗，主信息需有明确视觉强调。
#
# 用法：
#   source("scripts/plot_style.R")
#   p <- ggplot(...) + theme_mcup()
#   save_paper(p, "figures/fig1.png")
#   plot_confusion_matrix(cm, c("dent","hole","rusty"), file = "figures/cm.png")
#   plot_pr_curves(curves, file = "figures/pr.png")
#   plot_corr_heatmap(cor_mat, file = "figures/corr.png")
#   plot_feature_importance(names, values, file = "figures/imp.png")
suppressPackageStartupMessages({
  library(ggplot2)
  library(ragg)
})

FONT_PAPER <- "SimSun"
FONT_SANS  <- "SimHei"
```

```
CYCLE <- c("#1f77b4", "#ff7f0e", "#2ca02c", "#d62728")
NAVY  <- "#00007f"

# 逐条重要性条用的冷暖色（近似 matplotlib Spectral，自上而下红→蓝）
SPECTRAL <- c("#D53E4F", "#F46D43", "#FDAE61", "#FEE08B",
              "#E6F598", "#ABDDA4", "#66C2A5", "#3288BD")
spectral_n <- function(n) colorRampPalette(SPECTRAL)(n)

# 紧凑、干净的 M 杯主题
# legend: "br"=右下、"tr"=右上、"none"=不要图例
theme_mcup <- function(mode = c("paper", "sans"), base_size = 10, legend = "br",
                       title_family = NULL) {
  mode <- match.arg(mode)
  fam <- if (mode == "paper") FONT_PAPER else FONT_SANS
  lg <- switch(legend,
    br = list(pos = c(0.985, 0.02), just = c(1, 0)),
    tr = list(pos = c(0.985, 0.985), just = c(1, 1)),
    bl = list(pos = c(0.015, 0.02), just = c(0, 0)),
    right = list(pos = "right"),
    none = NULL,
    stop("legend 必须是 br / tr / bl / right / none")
  )
  th <- theme_bw(base_size = base_size, base_family = fam) +
    theme(
      panel.grid = element_blank(),
      panel.border = element_rect(colour = "black", linewidth = 0.5),
      panel.background = element_rect(fill = "white", colour = NA),
      plot.background = element_rect(fill = "white", colour = NA),
      axis.text = element_text(colour = "black", size = base_size - 1),
      axis.title = element_text(colour = "black", size = base_size - 0.5,
                                face = "bold"),
      axis.ticks = element_line(colour = "black", linewidth = 0.35),
      axis.ticks.length = unit(1.6, "pt"),
      plot.title = element_blank(),
      plot.tag = element_text(colour = "black", face = "bold",
                              size = base_size),
      strip.background = element_blank(),
      strip.text = element_text(colour = "black", face = "bold",
                                size = base_size - 0.5,
                                margin = margin(2, 1, 3, 1)),
      plot.margin = margin(4, 5, 3, 3),
      legend.background = element_blank(),
      legend.box.background = element_blank(),
      legend.key = element_blank(),
      legend.title = element_blank(),
      legend.text = element_text(size = base_size - 1.5),
      legend.key.size = unit(0.82, "lines"),
      legend.key.width = unit(1.15, "lines"),
      legend.spacing.y = unit(0.02, "cm"),
      legend.margin = margin(1.5, 2.5, 1.5, 2.5)
    )
  if (is.null(lg)) {
    th + theme(legend.position = "none")
  } else if (!is.null(lg$pos) && length(lg$pos) == 1 && lg$pos == "right") {
    th + theme(legend.position = "right",
               legend.justification = "center",
               legend.margin = margin(0, 0, 0, 4),
               legend.key.height = unit(0.6, "lines"))
  } else {
    th + theme(legend.position = "inside",
```

```
              legend.position.inside = lg$pos,
              legend.justification = lg$just)
  }
}

# 保存 300dpi PNG
save_paper <- function(plot, file, width = 6, height = 4, dpi = 300) {
  dir.create(dirname(file), showWarnings = FALSE, recursive = TRUE)
  agg_png(file, width = width, height = height, units = "in", res = dpi, background = "white")
  on.exit(invisible(dev.off()), add = TRUE)
  print(plot)
  invisible(file)
}

# 混淆矩阵：Blues + 每格数值（深色格白字），短类名不旋转、无留白
plot_confusion_matrix <- function(cm, classes, file = NULL,
                                  normalize = FALSE, width = 4.2, height = 3.6) {
  stopifnot(is.matrix(cm), nrow(cm) == ncol(cm), length(classes) == nrow(cm))
  df <- expand.grid(pred = classes, true = classes)
  df$n <- as.vector(t(cm))
  if (normalize) {
    df$value <- df$n / ave(df$n, df$true, FUN = sum)
    lab <- sprintf("%.2f", df$value)
  } else {
    df$value <- df$n
    lab <- as.character(df$n)
  }
  thresh <- max(df$value) / 2
  df$txt <- ifelse(df$value > thresh, "white", "black")
  angle <- if (max(nchar(classes)) <= 7) 0 else 35
  p <- ggplot(df, aes(pred, true, fill = value)) +
    geom_tile(colour = "white", linewidth = 0.45) +
    geom_text(aes(label = lab, colour = txt), size = 3.1) +
    scale_colour_identity() +
    scale_fill_gradient(low = "#F7FBFF", high = "#08306B",
                        guide = guide_colourbar(barwidth = unit(0.32, "cm"),
                                                barheight = unit(2.3, "cm"),
                                                frame.colour = NA)) +
    scale_x_discrete(expand = c(0, 0)) +
    scale_y_discrete(expand = c(0, 0), limits = rev(classes)) +
    labs(x = "预测标签", y = "真实标签") +
    coord_fixed() +
    theme_mcup(legend = "right") +
    theme(axis.text.x = element_text(angle = angle, hjust = if (angle == 0) 0.5 else 1),
          axis.title.x = element_text(margin = margin(t = 6)),
          axis.title.y = element_text(margin = margin(r = 6)))
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# PR 曲线：各类细线 + 总曲线深蓝加粗，图例带 AP，无边框
plot_pr_curves <- function(curves, file = NULL, width = 5, height = 4) {
  dfs <- lapply(names(curves), function(nm) {
    d <- curves[[nm]]
    data.frame(recall = d$recall, precision = d$precision,
               cls = nm, ap = d$ap, stringsAsFactors = FALSE)
  })
  df <- do.call(rbind, dfs)
  df$is_all <- tolower(df$cls) %in% c("all", "all classes", "micro", "总体")
  others <- setdiff(unique(df$cls[!df$is_all]), character(0))
  lev <- c(others, if (any(df$is_all)) "all classes")
```

```
  lab <- vapply(lev, function(k) {
    ap <- df$ap[df$cls == k][1]
    if (is.na(ap)) ap <- df$ap[df$is_all][1]
    if (k == "all classes") sprintf("all classes %.3f mAP@0.5", ap) else sprintf("%s %.3f", k, ap)
  }, character(1))
  df$grp <- factor(ifelse(df$is_all, "all classes", df$cls), levels = lev, labels = lab)
  col_map <- setNames(c(CYCLE[seq_along(others)], NAVY), lab)
  lw_map <- setNames(ifelse(lev == "all classes", 1.15, 0.55), lab)
  p <- ggplot(df, aes(recall, precision, colour = grp, linewidth = grp)) +
    geom_line() +
    scale_colour_manual(values = col_map) +
    scale_linewidth_manual(values = lw_map, guide = "none") +
    coord_cartesian(xlim = c(0, 1), ylim = c(0, 1.01), expand = FALSE) +
    labs(x = "Recall", y = "Precision", title = "Precision-Recall Curve") +
    theme_mcup(legend = "bl")
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# 相关性热力图：hot（负值深、正值浅）+ 全格数值
plot_corr_heatmap <- function(mat, file = NULL, width = 5.2, height = 4.6) {
  stopifnot(is.matrix(mat), nrow(mat) == ncol(mat))
  nm <- rownames(mat); if (is.null(nm)) nm <- paste0("V", seq_len(nrow(mat)))
  df <- expand.grid(var2 = nm, var1 = nm)
  df$value <- as.vector(t(mat))
  df$txt <- ifelse(df$value < -0.2, "white", "black")
  p <- ggplot(df, aes(var2, var1, fill = value)) +
    geom_tile() +
    geom_text(aes(label = sprintf("%.2f", value), colour = txt), size = 2.4) +
    scale_colour_identity() +
    scale_fill_gradientn(colours = c("black", "red", "orange", "yellow", "white"),
                         limits = c(-1, 1),
                         guide = guide_colourbar(barwidth = unit(0.3, "cm"),
                                                 barheight = unit(2.6, "cm"),
                                                 frame.colour = NA)) +
    scale_x_discrete(expand = c(0, 0)) +
    scale_y_discrete(expand = c(0, 0), limits = rev(nm)) +
    labs(x = NULL, y = NULL, title = "相关性热力图") +
    coord_fixed() +
    theme_mcup(legend = "right") +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# 特征重要性：横向降序、逐条冷暖色、末端数值
plot_feature_importance <- function(names, values, file = NULL,
                                    width = 6, height = NULL, top_n = 15) {
  ord <- order(values, decreasing = TRUE)[seq_len(min(top_n, length(values)))]
  df <- data.frame(name = factor(names[ord], levels = rev(names[ord])),
                   value = values[ord])
  df$col <- factor(seq_len(nrow(df)))
  h <- if (is.null(height)) max(3.0, 0.30 * nrow(df) + 0.9) else height
  p <- ggplot(df, aes(value, name, fill = col)) +
    geom_col(width = 0.68) +
    geom_text(aes(label = sprintf("%.1f", value)), hjust = -0.15, size = 2.5) +
    scale_fill_manual(values = spectral_n(nrow(df)), guide = "none") +
    scale_x_continuous(expand = expansion(mult = c(0, 0.10))) +
    labs(x = "Relative Importance", y = NULL, title = "特征重要性") +
    theme_mcup(legend = "none") +
    theme(axis.text.y = element_text(size = 8.5))
```

```
  if (!is.null(file)) save_paper(p, file, width = width, height = h)
  p
}
```

q1_metrics.py

```
"""Reusable continuous and ordered-proxy metrics for Q1 predictions."""

from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.metrics import (
    cohen_kappa_score,
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)


N_LEVELS = 7
METRIC_COLUMNS = [
    "r2", "mae", "rmse", "spearman", "qwk_proxy", "acc_proxy",
    "acc_pm1_proxy", "level_mae_proxy", "level_spearman_proxy",
]


def _safe_spearman(first: np.ndarray, second: np.ndarray) -> float:
    first_array = np.asarray(first, dtype=float)
    second_array = np.asarray(second, dtype=float)
    if first_array.size < 2 or second_array.size < 2:
        return float("nan")
    if np.ptp(first_array) == 0.0 or np.ptp(second_array) == 0.0:
        return float("nan")
    return float(spearmanr(first_array, second_array).statistic)


def _nanmean_without_warning(values: list[float]) -> float:
    array = np.asarray(values, dtype=float)
    return float("nan") if np.isnan(array).all() else float(np.nanmean(array))


def quantile_levels(
    values: np.ndarray,
    reference: np.ndarray,
    n_levels: int = N_LEVELS,
) -> np.ndarray:
    """Map continuous values to integer levels using reference quantiles."""
    value_array = np.asarray(values, dtype=float)
    reference_array = np.asarray(reference, dtype=float)
    if value_array.ndim != 1 or reference_array.ndim != 1:
        raise ValueError("values and reference must both be one-dimensional")
    if not np.isfinite(value_array).all() or not np.isfinite(reference_array).all():
        raise ValueError("values and reference must be finite")
    if n_levels < 2:
        raise ValueError("n_levels must be at least two")
    quantiles = np.linspace(0.0, 1.0, n_levels + 1)[1:-1]
    edges = np.unique(np.quantile(reference_array, quantiles))
    return np.digitize(value_array, edges).astype(int)
```

```
def ordered_proxy_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return seven-bin seven-bin diagnostic metrics for one target."""
    true_levels = quantile_levels(y_true, y_true)
    predicted_levels = quantile_levels(y_pred, y_pred)
    difference = np.abs(true_levels - predicted_levels)
    level_spearman = _safe_spearman(true_levels, predicted_levels)
    return {
        "qwk_proxy": float(
            cohen_kappa_score(true_levels, predicted_levels, weights="quadratic")
        ),
        "acc_proxy": float(np.mean(difference == 0)),
        "acc_pm1_proxy": float(np.mean(difference <= 1)),
        "level_mae_proxy": float(np.mean(difference)),
        "level_spearman_proxy": level_spearman,
    }


def target_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return continuous and ordered-proxy metrics for one sensory target."""
    true_array = np.asarray(y_true, dtype=float)
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    metrics = {
        "r2": float(r2_score(true_array, pred_array)),
        "mae": float(mean_absolute_error(true_array, pred_array)),
        "rmse": float(np.sqrt(mean_squared_error(true_array, pred_array))),
        "spearman": _safe_spearman(true_array, pred_array),
    }
    metrics.update(ordered_proxy_metrics(true_array, pred_array))
    return metrics


def matrix_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Macro-average all Q1 metrics over the sensory targets."""
    true_array = np.asarray(y_true, dtype=float)
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    if true_array.ndim != 2:
        raise ValueError("prediction matrices must be two-dimensional")
    rows = [
        target_metrics(true_array[:, position], pred_array[:, position])
        for position in range(true_array.shape[1])
    ]
    return {
        name: _nanmean_without_warning([row[name] for row in rows])
        for name in METRIC_COLUMNS
    }


def evaluate_prediction_matrix(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    target_codes: np.ndarray,
    experiment_id: str,
    evaluation: str,
) -> pd.DataFrame:
    """Evaluate every target and return one tidy row per target."""
    true_array = np.asarray(y_true, dtype=float)
```

```
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    if true_array.ndim != 2 or true_array.shape[1] != len(target_codes):
        raise ValueError("target_codes do not match the prediction matrix")
    return pd.DataFrame(
        [
            {
                "experiment_id": experiment_id,
                "evaluation": evaluation,
                "target_code": str(code),
                **target_metrics(true_array[:, position], pred_array[:, position]),
            }
            for position, code in enumerate(target_codes)
        ]
    )


def summarise_target_metrics(by_target: pd.DataFrame) -> pd.DataFrame:
    """Macro-average target metrics for each OOF evaluation."""
    summary = (
        by_target.groupby(["experiment_id", "evaluation"], sort=False)[METRIC_COLUMNS]
        .mean()
        .reset_index()
    )
    summary.insert(2, "n_targets", by_target["target_code"].nunique())
    summary.insert(3, "n_samples", 250)
    return summary
```

requirements.txt

```
numpy==2.2.6
pandas==2.3.3
scipy==1.15.3
scikit-learn==1.7.2
```

run_nnls.py

```
"""完整嵌套交叉验证与新样本推断；默认只显示帮助。"""
from pathlib import Path
import argparse
import json
import numpy as np
import pandas as pd
from stacking_runner import fit_outer_nnls
from q1_metrics import matrix_metrics
from baseline_models import empirical_cdf_grade

def load_tables(folder):
    x=pd.read_csv(folder/'1.csv');y=pd.read_csv(folder/'4.csv')
    for frame in [x,y]:
        if frame.beer.isna().any() or not frame.beer.is_unique:
            raise ValueError('beer 必须完整且唯一')
    if set(x.beer)!=set(y.beer):raise ValueError('样本主键不匹配')
    y=y.set_index('beer').loc[x.beer].reset_index()
    meta=['beer','beer_id','tasting_category_fine']
    if not x.tasting_category_fine.equals(y.tasting_category_fine):raise ValueError('风格不匹配')
    return x,x.drop(columns=meta).to_numpy(float),y.drop(columns=meta)

def main():
    parser=argparse.ArgumentParser(description='感官组收缩 NNLS：复算指标、完整嵌套验证或新样本推断')
```

```
    parser.add_argument('--attachments',type=Path)
    parser.add_argument('--assets',type=Path,default=Path(__file__).resolve().parent.parent/'数据与结果')
    parser.add_argument('--output',type=Path,default=Path('运行结果'))
    parser.add_argument('--execute',action='store_true',help='执行完整 10 次重复 3 折嵌套验证')
    parser.add_argument('--predict-chemistry',type=Path,help='对新化学表预测；仅以现有 250 款训练')
    parser.add_argument('--recompute-oof',type=Path,help='只由已有归档复算，不训练')
    args=parser.parse_args();args.output.mkdir(parents=True,exist_ok=True)
    if args.recompute_oof:
        a=np.load(args.recompute_oof,allow_pickle=False)
        rows=[{'evaluation':f'repeat_{r}',**matrix_metrics(a['y_true'],a['prediction'][r])} for r in
range(10)]
        rows.append({'evaluation':'mean_oof',**matrix_metrics(a['y_true'],a['prediction'].mean(0))})
        pd.DataFrame(rows).to_csv(args.output/'预测指标.csv',index=False)
        return
    if not args.execute and not args.predict_chemistry:parser.print_help();return
    if args.attachments is None:parser.error('需要--attachments 指向官方附件目录')
    chem,x,y_frame=load_tables(args.attachments);y=y_frame.to_numpy(float)
    dictionary=pd.read_csv(args.assets/'target_dictionary.csv').set_index('target_code')
    groups=dictionary.loc[y_frame.columns,'target_group'].astype(str).tolist()
    config=json.loads((Path(__file__).parent/'model_config.json').read_text())
    if args.predict_chemistry:
        new=pd.read_csv(args.predict_chemistry)
        feature_names=chem.drop(columns=['beer','beer_id','tasting_category_fine']).columns
        result=fit_outer_nnls(x,y,new.loc[:,feature_names].to_numpy(float),base_config=config,groups=group
s,scopes=['group'],shrinkage=.5,inner_splits=3,seed=20261023)
        prediction=result['predictions']['group']
        frame=pd.DataFrame(prediction,columns=y_frame.columns);frame.insert(0,'beer',new.beer)
        frame.to_csv(args.output/'连续感官预测.csv',index=False)
        grade=pd.DataFrame(empirical_cdf_grade(y,prediction),columns=y_frame.columns);grade.insert(0,'beer
',new.beer)
        grade.to_csv(args.output/'相对强度预测.csv',index=False)
        return
    splits=pd.read_csv(args.assets/'cross_validation_splits.csv')
    pred=np.full((10,len(x),len(y_frame.columns)),np.nan);folds=np.full((10,len(x)),-1,int);rows=[]
    for repeat in range(10):
        for fold in range(3):
            selected=splits[(splits.repeat==repeat)&(splits.fold==fold)]
            valid=np.sort(selected.row_position.to_numpy(int));train=np.setdiff1d(np.arange(len(x)),valid)
            if not len(valid):raise ValueError('缺少外层划分')
            expected=chem.iloc[selected.row_position].beer.to_numpy()
            np.testing.assert_array_equal(expected,selected.beer.to_numpy())
            result=fit_outer_nnls(x[train],y[train],x[valid],base_config=config,groups=groups,scopes=['gro
up'],shrinkage=.5,inner_splits=3,seed=20261023+repeat*10+fold)
            pred[repeat,valid]=result['predictions']['group'];folds[repeat,valid]=fold
            rows.append({'repeat':repeat,'fold':fold,'weights':result['parameters']['group']})
            print(f'完成重复{repeat+1}、折{fold+1}',flush=True)
    if not np.isfinite(pred).all():raise ValueError('折外预测未完整覆盖')
    np.savez_compressed(args.output/'模型折外预
测.npz',y_true=y,prediction=pred,mean_prediction=pred.mean(0),beer=chem.beer.to_numpy(),style=chem.tasting
_category_fine.to_numpy(str),target_code=y_frame.columns.to_numpy(str),fold_assignment=folds)
    (args.output/'模型权重.json').write_text(json.dumps(rows,ensure_ascii=False,indent=2))

if __name__=='__main__':main()
```

stacking_runner.py

```
from __future__ import annotations
import numpy as np
from sklearn.model_selection import KFold
from baseline_models import fit_predict_extra_trees, fit_predict_ridge_quantile, fit_predict_pls_quantile
from ensemble_models import fit_predict_hgb_multioutput
```

```
from q1_metrics import matrix_metrics
from nnls_stacking import fit_nnls_stacking
def make_inner_splits(n_samples,n_splits,random_state):
    return
list(KFold(n_splits=n_splits,shuffle=True,random_state=random_state).split(np.arange(n_samples)))

def _candidates(config: dict, model: str) -> list[object]:
    if model == "extra_trees":
        return [
            {"max_features": float(max_features), "min_samples_leaf": int(leaf)}
            for max_features in config["extra_trees"]["max_features"]
            for leaf in config["extra_trees"]["min_samples_leaf"]
        ]
    if model == "hgb":
        return list(config["hgb_profiles"])
    if model == "ridge_quantile":
        return [float(value) for value in config["ridge_alphas"]]
    if model == "pls2_quantile":
        return [int(value) for value in config["pls_components"]]
    raise ValueError(f"unsupported base base model: {model}")

def _fit_base(
    model: str,
    parameter: object,
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    config: dict,
    random_state: int,
) -> np.ndarray:
    if model == "extra_trees":
        values = dict(parameter)
        return fit_predict_extra_trees(
            x_train,
            y_train,
            x_validation,
            n_estimators=int(config["extra_trees"]["n_estimators"]),
            max_features=float(values["max_features"]),
            min_samples_leaf=int(values["min_samples_leaf"]),
            max_depth=config["extra_trees"]["max_depth"],
            random_state=random_state,
            n_jobs=1,
        )
    if model == "hgb":
        values = dict(parameter)
        return fit_predict_hgb_multioutput(
            x_train,
            y_train,
            x_validation,
            max_leaf_nodes=int(values["max_leaf_nodes"]),
            min_samples_leaf=int(values["min_samples_leaf"]),
            learning_rate=float(values["learning_rate"]),
            l2_regularization=float(values["l2_regularization"]),
            max_iter=int(values["max_iter"]),
            random_state=random_state,
        )
    if model == "ridge_quantile":
        return fit_predict_ridge_quantile(
            x_train,
            y_train,
            x_validation,
```

```
            alpha=float(parameter),
            random_state=random_state,
        )
    if model == "pls2_quantile":
        return fit_predict_pls_quantile(
            x_train,
            y_train,
            x_validation,
            n_components=int(parameter),
            random_state=random_state,
        )
    raise ValueError(f"unsupported base base model: {model}")

def _parameter_key(metrics: dict[str, float]) -> tuple[float, ...]:
    return (
        metrics["mae"],
        -metrics["r2"],
        -metrics["spearman"],
        -metrics["qwk_proxy"],
        -metrics["acc_pm1_proxy"],
    )

def fit_outer_nnls(x_train, y_train, x_validation, *, base_config, groups,
                   scopes, shrinkage, inner_splits, seed) -> dict:
    """Rebuild base-model OOF entirely inside outer training, then fit NNLS."""
    y = np.asarray(y_train, dtype=float)
    splits = make_inner_splits(len(y), int(inner_splits), int(seed))
    assignment = np.full(len(y), -1, dtype=int)
    for fold, (_, valid) in enumerate(splits):
        assignment[valid] = fold
    inner_predictions, outer_predictions, selections = [], [], {}
    for model_position, model in enumerate(base_config["base_models"]):
        candidate_rows = []
        for candidate_position, parameter in enumerate(_candidates(base_config, model)):
            prediction = np.full_like(y, np.nan)
            for fold, (train, valid) in enumerate(splits):
                prediction[valid] = _fit_base(
                    model, parameter, x_train[train], y[train], x_train[valid], base_config,
                    random_state=int(seed) + model_position * 100000 + candidate_position * 1000 + fold,
                )
            if not np.isfinite(prediction).all():
                raise RuntimeError("stacking base inner OOF is incomplete")
            candidate_rows.append({"parameter": parameter, "prediction": prediction,
                                   "metrics": matrix_metrics(y, prediction)})
        selected = min(candidate_rows, key=lambda row: _parameter_key(row["metrics"]))
        inner_predictions.append(selected["prediction"])
        outer_predictions.append(_fit_base(
            model, selected["parameter"], x_train, y, x_validation, base_config,
            random_state=int(seed) + model_position * 100000,
        ))
        selections[model] = {
            "parameter": selected["parameter"],
            "candidate_metrics": [{"parameter": row["parameter"], "metrics": row["metrics"]}
                                  for row in candidate_rows],
        }
    inner_stack, outer_stack = np.stack(inner_predictions, axis=-1), np.stack(outer_predictions, axis=-1)
    predictions, parameters = {}, {}
    for scope in scopes:
        fitted = fit_nnls_stacking(inner_stack, y, groups=groups, scope=scope, shrinkage=shrinkage)
        predictions[str(scope)] = fitted.predict(outer_stack)
        parameters[str(scope)] = {"weights": fitted.weights.tolist(),
```

```
                                "intercept": fitted.intercept.tolist(),
                                "global_weights": fitted.global_weights.tolist()}
    return {"predictions": predictions, "parameters": parameters, "base_selection": selections,
            "inner_fold_assignment": assignment, "inner_prediction_stack": inner_stack,
            "outer_base_stack": outer_stack}
```